

Model Deployment

When doing professional Machine Learning your users should be able to easily access your model. The convenient way to do that is through an API, as explained in the lecture.

For this assignment you need to show that you've successfully deployed your model as an API, are able to send a request, and receive a response. Your model should also have some documentation that defines what a request should look like and what a response should look like. The API can be hosted locally and doesn't need to be available to the open internet.

Your system should perform **above random guessing**. For classification this is fairly easy to figure out (accuracy > percentage of sample in most common class), but for regression this is not so easy.

If you have a regression task, compare the validation MAE/MSE/RMSE of your predictions against what it would be if you always predict the mean.

For this deadline please submit:

1. A short description of what you've already implemented. This helps your TA assess whether things are on track.
2. **Evidence that you achieve above random guessing model performance on your validation data.** This can be your **validation** accuracy vs. random guess accuracy.
3. A screenshot of an API call made, and the subsequent response. This can be done with curl (CLI tool: <https://curl.se/>) or Postman (GUI tool: <https://www.postman.com/>), or a method of your choice. You should show both the request, and the successful response.
4. A screenshot of the API documentation for your API as demonstrated in the lecture. This should clearly indicate what the request should look like, and what the response should look like. The quality of your documentation will affect your grade.
5. **A way for us to access your github repo (important!).** This can be a link if your repository is public (recommended), or you can upload a .zip of your main branch. Your repo should have a [readme.md](#) that documents how to install dependencies and launch the API. **Make sure this actually works. You can also arrange this with your TA before the deadline!**
6. A statement of what each group member contributed.

If you're working with unusual data it may be difficult to send data through curl or Postman. Instead you may use the FastAPI Testclient (<https://fastapi.tiangolo.com/tutorial/testing/>).

Grading information

Your grade will be based on the following criteria. Depending on the quality of your work it can receive anywhere between full and no points.

- Successful API implementation (2 points)
- Appropriate preprocessing/postprocessing (2 points)
 - Returning class IDs is bad, return the name
 - Requiring the user to do normalization is bad, do it for them
 - Resizing images or turning them to grayscale is nicer than not doing it.
- RESTfulness of your API (3 points)
 - As explained in the lecture
- Proper documentation with expected data types (3 points)
 - Both expected input/output json (e.g. from Pydantic) and natural language to explain how to use endpoints
 - A well explained and well documented API is nice. A few quick words about e.g. “this classifies digits” is less nice.
 - This includes a good [README.md](#) with complete instructions.
- Error handling and HTTPExceptions (1 point)
 - Consider what might go wrong, and implement HTTPExceptions for those. There’s typically at least some bad inputs a user can give that you can catch. Think about what those might be.

We subtract 3 *points* if model not better than random guessing